Supplementary Technical Documentation

Multilingual LLM-as-a-Judge Reliability: Diagnosis, Selective Verification, and Targeted Adaptation

Bachelor's thesis, Çağla Ece Azizoğlu, Technical University of Munich, 2026

This document reproduces, unchanged, the full technical appendix (Appendices A–J) of the thesis as built on 29 October 2026. The submitted thesis contains a condensed appendix that keeps the parts referred to from the main text; the material here documents the remaining details: runtime settings, parsing and recovery rules, generation-limit diagnostics, the complete Korean preference-experiment tables, the verifier, bridge and translation-repair protocols, the qualitative case records and the evidence-archive guide.

Page numbers, and appendix section and table numbers inside these pages, follow that full build. References to Chapters 1–9, Tables 1–20 and Figures 1–6 point to the main text of the submitted thesis, whose numbering is unchanged.

A. Runtime Prompts

This appendix reproduces the exact prompts used in the runs. The evidence index (Appendix J.1) links each prompt to its script. Braces around JSON objects are literal characters of the prompt, and Python template escaping has been removed. Named placeholders stand for the complete text inserted at run time. The six templates match the prompts in the run scripts.

A.1. Holistic Judgements

Q1 shows the summary alone, while Q2 and Q3 show the source and the summary. Each prompt is a single user text message passed through `processor.apply_chat_template` with `add_generation_prompt=True`.

A.1.1. Q1: Comprehensibility

```
You are evaluating the quality and comprehensibility of a summary.

Judge ONLY the linguistic quality and comprehensibility of the summary itself.
Do NOT judge whether the summary is factually supported by an article.

Consider:
- grammatical correctness
- fluency
- clarity
- coherence
- repetition or broken phrasing
- whether the summary is easy to understand

Use exactly one of these scores:

0 = Very poor: largely incomprehensible or seriously broken.
25 = Poor: understandable only with substantial difficulty; major quality
↪  problems.
50 = Fair: understandable overall, but with noticeable fluency, grammar,
↪  clarity, coherence, or repetition problems.
75 = Good: clear and generally fluent, with only minor problems.
100 = Excellent: fully comprehensible, clear, coherent, grammatical, and fluent.

Evaluate only the summary below.

Summary:
{summary}

Return ONLY valid JSON in exactly this form:
```

```
{"score": 0, "reason": "brief explanation"}

The score MUST be exactly one of:
0, 25, 50, 75, 100
```

A.1.2. Q2: Factual Attribution

```
You are evaluating factual attribution.

Task:
Given a source document and a summary, judge how well the factual claims in the
↪  summary are supported by the source.

Use only the source document. Do not use outside knowledge.

Scoring scale:
100 = The summary is fully supported by the source. All important factual claims
↪  are attributable to the source.
75 = The summary is mostly supported, with only minor unsupported or imprecise
↪  details.
50 = The summary is partially supported, but it contains some important
↪  unsupported, vague, or overgeneralized claims.
25 = The summary has limited support. Most important claims are unsupported or
↪  only weakly related to the source.
0 = The summary is not supported by the source, contradicts the source, or
↪  describes a different event.

Pay special attention to:
- named entities
- roles
- who did what to whom
- dates
- numbers
- locations
- causal relations
- outcomes
- whether the summary adds information not in the source

Return valid JSON only:
{
  "score": 0,
  "reason": "short explanation"
}

SOURCE:
{source}
```

```
SUMMARY:
{summary}
```

A.1.3. Q3: Overall Summary Quality

```
You are evaluating the informativeness of a summary with respect to its source
↪  article.

Judge how well the summary captures the important information from the article
↪  accurately, clearly, and concisely.

Consider:
- whether the most important information is included
- whether important content is omitted
- whether the included information accurately reflects the article
- whether the summary is focused rather than filled with irrelevant detail
- whether it provides a useful concise representation of the article

Use exactly one of these scores:

0 = Very poor: fails to capture the important information or is seriously
↪  misleading.
25 = Poor: captures little of the important information and has substantial
↪  omissions or problems.
50 = Fair: captures some important information but misses or distorts meaningful
↪  content.
75 = Good: captures most important information accurately with only minor
↪  omissions or weaknesses.
100 = Excellent: captures the important information accurately, clearly, and
↪  concisely.

Source article:
{source}

Summary:
{summary}

Return ONLY valid JSON in exactly this form:
{"score": 0, "reason": "brief explanation"}

The score MUST be exactly one of:
0, 25, 50, 75, 100
```

A.2. Translation and Structured Verification

One instance of `google/gemma-3-4b-it` translates the source, translates the summary, extracts claims from the translated summary and verifies each claim against the translated

source, in this order. In the translation prompt, `{text_type}` is replaced by `source document` or `candidate summary`. The instruction asks the model to keep names, dates, numbers and roles exactly as they are, but it cannot guarantee that the model does so.

A.2.1. Translation

```
Translate the following {text_type} into English.

Rules:
- Preserve all names, dates, numbers, quantities, places, organizations, and
↪  roles exactly.
- Do not summarize.
- Do not add information.
- Do not omit factual details.
- Return only the English translation, with no commentary.

Text:
{text}
```

A.2.2. Atomic-Claim Extraction

```
You are decomposing a candidate summary into atomic factual claims.

Rules:
- Extract only factual claims stated in the summary.
- Split compound claims into separate atomic claims.
- Each claim should be checkable against a source document.
- Do not add information that is not in the summary.
- Do not judge whether the claims are true yet.
- If the summary contains no factual claim, return an empty list.

Candidate summary in English:
{summary_en}

Return valid JSON only:
{
  "claims": [
    "claim 1",
    "claim 2"
  ]
}
```

A.2.3. Single-Claim Verification

```
You are verifying ONE atomic factual claim against a source document.

Your job is strict factual attribution.

First, check the claim normally.
Then, before giving the final label, silently ask yourself:
```

```
"Am I overestimating support because the source is only topically related, only
↪  partially supports the claim, or supports a weaker version of the claim?"

A claim is fully_supported ONLY IF:
- the exact same main entity is present in the source,
- the exact same action, role, relation, or event is present in the source,
- all numbers, dates, quantities, locations, and names match,
- no important detail is missing,
- the evidence directly supports the full claim, not just a related weaker
↪  claim.

Use these labels:
- fully_supported: the complete claim is directly supported.
- partially_supported: some parts are supported, but an important detail is
↪  missing, weaker, vague, or uncertain.
- unsupported: the source is related but does not support the exact claim.
- contradicted: the source says something incompatible with the claim.

Important:
If you are uncertain between fully_supported and partially_supported, choose
↪  partially_supported.
If you are uncertain between partially_supported and unsupported, choose
↪  unsupported.

English source document:
{source_en}

English claim:
{claim_en}

Return valid JSON only:
{
  "label": "fully_supported/partially_supported/unsupported/contradicted",
  "evidence": "short exact evidence span from the English source, or empty
  ↪  string",
  "missing_or_weaker_detail": "what is missing, weaker, or wrong; empty if
  ↪  none",
  "reason": "brief explanation"
}
```

A.3. Korean TEST50 Prompt

Braces mark the input slots that are filled at run time. The English translation is for readers only and was not used in the experiment. Appendix D.3 describes how the prompt was serialized and scored.

다음 기사와 두 개의 요약을 읽으세요.

기사:
{article}

요약 A:
{summary_a}

요약 B:
{summary_b}

기사의 내용에 더 정확하게 근거한 요약을 선택하세요. 문체, 길이, 표현의 자연스러움이 아니라 사실적 정확성만 판단하세요.
A 또는 B 중 하나만 답하세요.

English translation for readers Read the following article and two summaries.

> Article: {article}
> Summary A: {summary_a}
> Summary B: {summary_b}

Choose the summary that is more accurately grounded in the article. Judge factual accuracy only, rather than style, length or naturalness of expression. Answer with only one of A or B.

B. Generation, Parsing, and Recovery

This appendix documents how the development sample was drawn, how the model outputs were generated and parsed, and how failed outputs were handled.

B.1. Evaluation Sample

The release contains minimum, average and maximum score columns for each dimension. In the released file, the Q2 and Q3 minima and maxima take only the values 0 and 1, while the Q1 extrema take the values 0, 0.5 and 1. This fits binary ratings for Q2 and Q3 and normalized three-level ratings for Q1, but it does not reveal how the original choices were converted. The paper documents three-level choices for Q1 and binary choices for Q2 and Q3. Neither the paper nor the official README specifies the numeric mapping used in the release.

Of the 31,500 released Q2 averages, 729 (2.3%) take values that cannot result from an unweighted mean of exactly three binary ratings. Twenty-two such rows occur in the 900-example evaluation sample. The actual number of ratings per item and their weighting therefore remain unclear. I use the released averages unchanged and only multiply them by 100. The reported metrics need only these averages, not a reconstruction of individual ratings. Their interpretation as attribution judgements, however, depends on how they were annotated and aggregated. The discrepancy points to incomplete documentation of the release, not to wrong labels.

The notebook that produced the 900-example development input first removes rows that lack a source document, a summary or any of the three human averages. It then samples 20 examples per language with `random_state=42`. Applied to the full release, the filter removes no rows, and drawing 20 rows per language with this seed reproduces all 900 rows of the evaluation sample exactly. The same input, with 20 examples for each of the 45 languages, is reused in all Q2 interventions.

B.2. Runtime and Input Construction

The main mFACE runner uses `google/gemma-3-4b-it` in bfloat16, with automatic device mapping and `do_sample=False`. Its output limit is 256 new tokens. The complete source and summary strings are inserted without truncation. The translated verifier uses output limits of 4,096 new tokens for source translation, 1,024 for summary translation, 512 for extraction and 512 for each verification, with `temperature=None` and `top_p=None`. These limits only cap the output length. The script does not check translation fidelity and does not detect truncated outputs.

The Q1 and Q3 runners are derived from the Q2 runner and use the same 900-example development input. The verifier input is built directly from the repaired Q2 baseline. The Q2 output field `row_id` is a local DataFrame index and not necessarily the global mFACE identifier. The separate 900-example sample from earlier structured-judging runs is not used in these runs.

All output files were checked against the input: each of the 900 examples appears exactly once in every file, and the human scores and baseline judgements agree across files.

B.3. Q2 Parsing and Score Recovery

The parser first tries to read the whole response with `json.loads`. If that fails, it tries a greedy brace-delimited object. The original `valid_json` flag requires both a parsed score and a score in $\{0, 25, 50, 75, 100\}$. It therefore combines parsing with anchor compliance and does not measure JSON syntax alone. Parsed scores between anchors are kept as they are, not rounded.

The final Q2 file contains 883 true flags, 16 parsed scores between anchors and one malformed JSON response with a score of 75. Only missing numerical scores are repaired, using the pattern

```
"score"\s*:\s*([0-9]+(?:\.[0-9]+)?)
```

The original raw response and validity flag are kept. Row 779 supplies the recovered score of 75, so all 900 rows have a usable score.

The 883 true flags give a parser-and-anchor compliance of 98.1%. The anchor-only robustness subset has $n = 884$ instead, because it adds row 779 and excludes the 16 scores between anchors. The two numbers come from different filters. The main results use all 900 numerical scores.

B.4. Q1 Recovery and Mean Correction

One Q1 response (row 242, Igbo) states a score of 75 but has malformed quoting in its reason, so the parser initially missed it. Recovering this stated score gives numerical scores for all 900 rows, including two scores of 95 that lie between the anchors. All Q1 results in this thesis use these 900 rows.

B.5. Claim Parsing Fallbacks

Failed outputs stay in the scores, and the script does not retry failed generations. An extraction output that cannot be parsed falls back to an empty claim list, which receives a claim score of zero. A verification response that cannot be parsed defaults to `unsupported`.

The full 900-row output contains 2,489 extracted claims. Extraction cannot be parsed for two rows: Igbo row 241 returns a request for input, and Korean row 332 contains malformed quotation marks. Ukrainian row 792 returns a valid empty list. All three rows receive a claim score of zero. Verification cannot be parsed for 29 claims across 27 rows, and each of these claims falls back to `unsupported`. The recorded labels are 1,235 fully supported, 597 partially supported and 657 unsupported claims, including the 29 fallbacks. No label is missing, unrecognized or `contradicted`.

Thirty rows contain an empty extraction or at least one verifier parse failure. The selective rule changes the baseline score in 14 of them. Ten of these move closer to the human reference and four move further away, while the other 16 rows keep the same error.

B.6. Outputs Near the Generation Limit

The original token counts and stopping reasons were not saved. Re-tokenizing the stored outputs with the pinned tokenizer flags seven outputs within eight tokens of their limit: one source translation, one summary translation and five verification responses. No holistic response (maximum 180 of 256 tokens) and no claim extraction (maximum 240 of 512) comes close. The Telugu source translation of row 710 and the Gujarati summary translation of row 193 end in repetition loops. Four of the verification responses end in incomplete JSON after reproducing long non-English passages and fall back to `unsupported`. The fifth, in row 736, is complete and parses, so nearness to the limit does not by itself indicate truncation. The exact stopping reasons cannot be recovered (Appendix J.2).

C. Development Diagnostics

The results in this appendix explain why particular methods were chosen. They are not independent evidence that these methods generalize.

The resource groups use XL-Sum training counts, obtained by counting the training split of each language in the Hugging Face release (`csebuetnlp/xlsum`).

C.1. Resource Proxies and Tokenizer Fragmentation

Preliminary exploration of all three criteria motivated the focus on Q2; it is not part of the reported evidence.

The resource mapping contains 18 high-resource, 15 medium-resource and 12 low-resource languages, and its thresholds reproduce all 45 group labels. Recomputing Q2 rank agreement within these groups reproduces the resource tables in Chapters 4 and 5.

An exploratory analysis relates tokenizer fragmentation, measured as the average number of tokens per character of each language's 20 development sources, to the language-level Spearman correlation of the initial Q2 judgement. Across the 45 languages, the Pearson correlation is –0.327. In single-predictor regressions with standardized predictors, tokens per character gives $R^2 = 0.107$, compared with 0.013 for log(training count + 1). Fragmentation is thus more strongly associated with agreement in this sample, but the association is modest, and these in-sample associations do not show that fragmentation causes weaker judging. The correlations and regressions reproduce from the saved features in the evidence archive; the token counts themselves cannot be regenerated (Appendix J.2).

C.2. Translation Development Subsets and Method Selection

On dev10, MAE is 36.67 for the initial judgement, 32.50 for capped claims in the original language and 13.75 for translated capped claims. On dev50, MAE is 38.50 for the initial judgement and 28.03 for translated capped claims.

Dev10 is balanced across the five baseline error groups, with two rows from each, and dev50 contains ten rows from each. Every dev10 row also appears in dev50, and both subsets lie entirely within the final 900-row sample. These subsets are not independent held-out tests.

The evidence archive contains the scripts, output tables and logs of these development runs. The selective rule was compared with an earlier combination rule on the same development sample, so its development results are conditional on this choice.

D. Korean Preference Experiment

This appendix documents the controlled Korean preference experiment of Chapter 6: its data, training settings, evaluation protocol, detailed results and reproduction checks. The supervised NLI verifier is documented separately in Appendix E.

Korean is one focused setting for this design. In the 20 Korean examples of the mFACE Q2 evaluation, the initial holistic judgement has a diagnostic Spearman correlation of about –0.594 and an MAE of 53.33. Selective structured verification changes these to about 0.268 and 44.38. With only 20 examples, these values describe this sample, not the language in general.

D.1. Data Construction and Split

The preference data come from Korean XL-Sum sources (Hasan et al., 2021), and no mFACE example is used for training. To exclude exact overlap with the Korean part of mFACE, I normalize each source by collapsing consecutive whitespace and trimming both ends, compute a SHA-256 hash, and drop every XL-Sum article whose hash matches a Korean mFACE source. This removes exact overlap after normalization.

The split procedure excludes the sources used earlier in development, removes normalized duplicates, shuffles the remaining sources, and assigns separate sets of 50 test sources and 50 training sources, each with 50 distinct articles. There is no overlap between training and test, and no exact overlap with the 900-row mFACE sample.

Each pair contains one Korean article and two candidate summaries. One candidate is grounded in the source. The other is closely related but contains one targeted change, a *near-miss corruption*. The corruption families concern quantities, relations, scope and time, which links them conceptually to the audit categories.

Each pair records which candidate the source supports better. These labels are *construction-based*. They follow from the corruption design and a review of compatibility with the source, not from independently collected human preferences. I reviewed the final pairs against their articles, balanced the design, and fixed the data before inspecting any final test output.

Each training pair appears in both candidate orders, which gives 100 presentations. TEST50 also has two orientations per pair, but only 50 independent units. Table 21 shows that the preferred label and the severity are balanced, and Table 23 gives the family sizes and results.

The design has three limits. The overlap checks exclude exact duplicates but not near-

Table 21 Controlled Korean preference-data design.

Property	Training	TEST50
Semantic pairs	50	50
Preferred A/B label balance	25/25	25/25
Major/minor severity balance	25/25	25/25
Both candidate orders	Yes	Yes

duplicates in meaning, overlap through translation or exposure during pretraining. The corruption families differ in size, so their results are descriptive only. And AI-assisted construction may favour regularities that the evaluated model recognizes, which my review cannot exclude. The transfer analyses in Appendix D.5 test how far the result reaches beyond the constructed distinctions.

D.2. Training Settings

Table 22 lists the main settings of the final run. The run also uses bfloat16 and gradient clipping at a maximum norm of 1.0. Five epochs over 100 presentations give 500 optimizer steps.

Table 22 Main settings of the final Korean preference-optimization run.

Setting	Value
Preference objective	DPO
DPO β	0.1
Epochs	5
Learning rate	1×10^{-5}
Maximum sequence length	3072
LoRA rank	8
LoRA α	16
LoRA dropout	0.05
Effective batch size	1 presentation
Gradient accumulation	None
Optimization steps	500

The adapter covers seven attention and feed-forward projection modules, with no bias adaptation. The manual DPO loop uses AdamW (Loshchilov and Hutter, 2019) over the trainable parameters, with the literal labels A and B as answer targets. Reference scores come from the same model with its adapters disabled and are computed without gradients. The input is the stripped prompt followed by a newline, with special tokens disabled. Answer tokens

are always kept, and the prompt is truncated from the left when necessary to fit the total limit of 3,072 tokens. Training sums the log-probabilities of the answer tokens using a float32 log-softmax, and each epoch shuffles the data.

The average training loss in epoch 5 is 0.0553. It is a training diagnostic and is not used as evidence for RQ4a. This adapter is separate from the supervised verifier trained later.

D.3. Evaluation Protocol

The base and adapted models receive identical Korean source–candidate pairs, using the prompt in Appendix A.3. The original evaluator tokenizes the stripped prompt followed by a newline with `add_special_tokens=False`. It does not apply the native chat template used for mFACE. Each pair is evaluated in its original orientation and with the candidates swapped. A stable preference requires the A/B answer to flip while the preferred summary stays the same, which addresses known position effects (Section 2.2).

The frozen protocol fixed two evaluation modes before any final output was inspected.

Strict free generation (primary) The model generates at most four new tokens with deterministic decoding. Decoded whitespace is stripped, and an output is valid only if it is exactly `A` or exactly `B`. Every other output counts as incorrect.

Label-logprob accuracy (secondary) The evaluator tokenizes the literal labels `A` and `B` separately, with special tokens disabled. It sums the conditional log-probabilities of each label's tokens using a float32 log-softmax, without normalizing for sequence length. The label with the higher probability counts as the model's choice. An exact tie yields no choice and counts as incorrect. This measure does not require the model to produce an exact one-character answer.

Both rules apply to the plain-text serialization, not to the chat template used for mFACE. The original and swapped presentations are two views of the same 50 pairs, not 100 independent pairs. The secondary measure does not replace the primary endpoint.

D.4. Results by Corruption Family and Severity

Table 23 breaks down label-logprob accuracy in the original orientation by corruption family. With 8 to 18 pairs per family, it shows where the changes occur, but it does not test whether one family is easier than another.

Accuracy rises in all four families. The largest gain is for temporal/scope corruptions, from 38.89% to 94.44%. Numeric/quantitative corruptions remain the hardest after adaptation,

Table 23 Original-orientation TEST50 label-logprob accuracy by corruption family.

Corruption family	*n*	Base	Adapted
Numeric/quantitative	8	25.00%	62.50%
Relational/policy/status	13	53.85%	84.62%
Scope/evidence overstatement	11	90.91%	100.00%
Temporal/scope	18	38.89%	94.44%

rising from 25.00% to 62.50%. Severity shows the same direction. Accuracy rises from 68% to 96% for the 25 major corruptions and from 36% to 80% for the 25 minor ones. The subgroups are small, so the differences between families and between severity levels are uncertain.

D.5. Robustness and Transfer of the Preference Adapter

This section gives the full analyses summarized in Section 6.2.

D.5.1. Original Free-Generation Diagnostic

Chapter 6 keeps the pre-specified primary outcome. In the original order, the difference is +46 percentage points, with a paired bootstrap interval of $[+32, +60]$. The exact McNemar comparison (McNemar, 1947) counts 23 pairs that only the adapter gets right and none that only the base model gets right ($p < 10^{-6}$). These statistics include format failures and do not isolate factual choice. Table 24 separates valid answers, strict accuracy and semantic consistency under the original plain-text prompt.

Table 24 Original free-generation diagnostics under plain-text serialization. Invalid responses count as incorrect; these outcomes do not isolate factual choice.

Outcome and evaluation unit	Base	DPO adapter
Original order: 50 semantic pairs		
Valid exact A/B responses, *n*/50	0/50	26/50
Correct responses, *n*/50	0/50	23/50
Accuracy counting invalid responses as wrong (%)	0	46
Both orders: the same 50 pairs, 100 presentations		
Accuracy counting invalid responses as wrong (%)	0	47
Same semantic choice in both orders, *n*/50	0/50	19/50

The two orders are dependent presentations of each pair. Invalid responses cannot yield a consistent semantic choice. The last row expresses the swap-consistency rates of 0% and 38% as counts. Accuracy among valid original-order responses is undefined for the base model and 23/26 = 88.46% for the adapter.

D.5.2. Chat-Serialization Robustness

After seeing the original results, I reran the same fixed TEST50 pairs with the base model and the final adapter, under both the original plain-text serialization and the native chat serialization. The prompt wording, candidate order, deterministic decoding and four-token budget stay the same. This is a post-hoc robustness check on a test set that had already been inspected. Table 25 reports the results under the chat template.

Table 25 Strict generation under native chat serialization. Each orientation contains the same 50 semantic pairs.

Measure	Base	Adapted
Original-order accuracy (%)	66	74
Swapped-order accuracy (%)	64	84
Invalid outputs (n/100)	0/100	0/100
Semantic swap consistency (%)	30	58

Under the chat template, the base model returns a valid exact label for every presentation. Its original strict accuracy of 0% therefore does not mean that it cannot tell the candidates apart. Native formatting removes the observed format failure at this token budget. The adapter keeps a descriptive advantage of eight percentage points in the original order and twenty after swapping. Averaged over both orientations, accuracy is 65% for the base model and 79% for the adapter. Native label-logprob accuracy is 64% against 74% in the original order and 66% against 82% after swapping.

In the native original order, four pairs improve and none get worse. After swapping, ten improve and none get worse. The original-order contrast therefore rests on only four discordant pairs, which gives a two-sided exact paired test of $p = 0.125$. The small-sample bootstrap interval is positive, but the difference should not be called significant. Appendix D.6 describes how that interval was computed and adds a calculation over both orientations.

The breakdown by error type is also narrower than the label-probability result might suggest (Table 26). In the native original order, the improvements come from two scope/evidence cases, one relational/policy/status case and one temporal/scope case. Numeric/quantitative accuracy stays at 5/8. All four original-order corrections are major corruptions. Accuracy changes from 15/25 to 19/25 for major cases, while minor cases stay at 18/25. After swapping, the changes are distributed differently. These small, order-dependent groups do not show which factual mechanism the adapter has learned.

D.5.3. Transfer to Non-Korean Attribution Scoring

The transfer check removes Korean from the verified 900-row input, which leaves 880 examples across 44 languages, with 20 per language. I ran the unchanged base model and the final adapter again with the same pinned checkpoint, processor, native chat serialization, Q2 prompt, bfloat16 precision and deterministic limit of 256 new tokens. The adapter is thus the only difference between the two conditions. The original 900-row baseline is not used for

Table 26 Native-chat strict correct counts by corruption family. Original and swapped columns describe the same semantic pairs.

Family	*n*	Original		Swapped	
		Base	Adapted	Base	Adapted
Numeric/quantitative	8	5	5	6	7
Relational/policy/status	13	8	9	8	11
Scope/evidence overstatement	11	7	9	9	9
Temporal/scope	18	13	14	9	15

comparison, and no retraining or adapter selection took place.

The primary contrast is the difference in pooled Spearman correlation between the adapted and base models, computed on shared numerical outputs. All 880 scores are available in both conditions, so no rows are excluded. A paired language-cluster bootstrap resamples the 44 languages, keeping both model scores together, with 2,000 draws. The secondary error measures and subgroup analyses are exploratory. Table 27 reports the results.

Table 27 Exploratory non-Korean mFACE Q2 transfer, 880 matched examples.

Measure	Fresh base	Adapted
Spearman	0.2706	0.2790
MAE	34.12	34.05
RMSE	41.52	41.46
Signed bias	+17.71	+18.04
Extreme over-scoring	22.05%	22.05%

The primary difference is +0.0084, with a 95% percentile interval of [–0.0081, +0.0238]. The check shows no clear overall transfer gain, but it cannot rule out a small effect. MAE falls by 0.068 points on the 0–100 scale, and the positive signed bias rises by 0.330 points. Only 53 scores change. Of these, 29 move closer to the human reference and 24 move further away, while the other 827 stay the same.

Parser-and-anchor compliance is 861/880 for the base model and 859/880 for the adapter. One base-model score needed numeric recovery, and none of the adapter's scores did. No output reached the generation limit.

This check changes both the language and the task, from constructed Korean pairwise choices to natural multilingual attribution scores. It cannot separate a limit on language transfer from a limit on task transfer. The benchmark had also already informed the thesis, so this is an exploratory check within that benchmark, not an external validation.

D.5.4. Transfer to Human-Labelled Korean Inference

The controlled TEST50 preferences depend on how the corruptions were built and on my review. To see whether the adapter's advantage extends beyond that procedure, I evaluate it on a pairwise reformulation of the published KLUE-NLI development set (Park et al., 2021), which I defined myself. For each of 1,000 premises, the two candidates are the original entailment and contradiction hypotheses. The candidate sentences and majority-vote labels, each supported by at least three of five annotator votes, are kept verbatim, and neutral hypotheses are excluded. The correct candidate comes first in 500 pairs and second in the other 500. This is a separate human-labelled task, not an official three-class KLUE evaluation.

I fixed the protocol before generating these outputs, but only after earlier transfer results had raised the question. The primary endpoint is strict free-generation accuracy in the original candidate orientation with the premise provided, and invalid outputs count as incorrect. The same pinned base checkpoint and the fixed DPO adapter are compared under native chat serialization, with a swapped presentation and a premise-free condition as secondary controls.

Primary accuracy rises from 862/1,000 (86.2%) to 872/1,000 (87.2%). Fifteen pairs improve and five get worse, while 857 are correct for both models and 123 are wrong for both. The paired 95% percentile interval for the difference is [+0.2, +1.9] percentage points, and the two-sided exact paired test gives a nominal $p = 0.041$. The effect is small; the interval ignores possible clustering of premises within documents. In this orientation the adapter has three invalid outputs and the base model none, so format compliance does not explain the gain.

With swapped candidates, strict accuracy is 84.2% for the base model and 85.4% for the adapter. Without the premise, accuracy in the original orientation is 54.4% and 55.2%. Both models benefit from the source information, but the adapter's additional benefit is only +0.2 percentage points, with an interval of [–1.5, +1.9]. The premise-omission control therefore gives no evidence that the adapter grounds its choices better, and the published data may overlap with the model's pretraining data.

D.6. Reproduction Checks

The protocol, test inputs and adapter configuration of the Korean experiment match their recorded checksums, and the reported results can be recomputed from the saved outputs without rerunning the model. For the original plain-text evaluation, these saved outputs come from a later rerun that reproduces the original aggregate results.

The frozen robustness analysis bootstraps the original orientation only (+8 percentage points, [+2, +16]). Because the written plan had included both orientations, a supplementary analysis averages them within each pair and resamples the 50 pairs (+14 points, [+8, +20]); it does not replace the primary result.

E. Supervised NLI Verifier and Downstream Bridge

This appendix documents the supervised NLI verifier and the downstream bridge of Section 6.3. The verifier is trained separately from the DPO adapter in Appendix D.

E.1. Data Selection

The supervised data come from the official KLUE training release. An eligible premise group keeps one example of each class, and each gold label must have at least three of five annotator votes. Fixed genre quotas select the internal test set first, the validation set second and the training set last. All original strings and labels are kept.

After normalization, premise and hypothesis texts do not overlap between the three splits of 3,000, 600 and 600 examples, or with the public development set inspected earlier. Earlier source pools of this project were also checked for premise substrings, which excluded no otherwise eligible group. Paraphrases and exposure during pretraining cannot be ruled out.

E.2. Training and Checkpoint Selection

A new text-only LoRA adapter covers the q, k, v, o, gate, up and down projections, with rank 8, α = 16, dropout 0.05 and no bias adaptation. The native chat serialization and the three single-token labels A, B and C are fixed. The loss covers only the answer token and the native end-of-turn token. AdamW uses a learning rate of 5×10^{-5}, weight decay 0, $\epsilon = 10^{-8}$, gradient clipping at 1 and a constant learning rate, with a microbatch of 1 and 8 accumulation steps. Two epochs give 750 optimizer updates. No input is truncated, and the longest training sequence has 221 tokens.

The validation negative log-likelihood over the three label tokens is 3.524 at epoch 0, 0.493 at epoch 1 and 0.404 at epoch 2, so epoch 2 was selected before the internal test. Epoch 1 has a higher validation accuracy (90.00% against 88.67%), but accuracy was not the selection criterion, and the test set was not used to revise the choice. The internal bootstrap resamples the 200 premises, keeping the three hypotheses of each premise together, with 2,000 draws.

E.3. Bridge Protocol and Audit

The bridge protocol was frozen after the internal NLI result and before bridge inference. It reuses the exact original 900-row verification output with its 2,489 claims. The full available translated source is inserted without new truncation or retrieval. The longest prompt has

4,217 tokens, and 2,482 prompts are longer than the longest training sequence. Neutral is treated as missing entailment, not as partial support or a 50-point label. The priority of the contradiction cap over the neutral cap was fixed before any output existed. The three rows without claims keep a capped score of zero.

Both bridge conditions use the same inputs and a single model instance, with the new adapter disabled or enabled. Next-token logits are restricted to A, B and C, so the bridge does not test whether the model follows the answer format in free generation. All reported bridge results can be recomputed from the saved label decisions.

Section 6.4 reports and interprets the results. A supplementary post-hoc audit adds intervals for the differences in extreme errors: for SV-NLI minus SV, extreme under-scoring changes by [+1.33, +4.33] percentage points and extreme over-scoring by [–2.67, –0.22]. No threshold, checkpoint or subset was tuned afterwards.

F. Translation Repair Protocol

This appendix documents the translation repair evaluated in Section 5.4.

F.1. Selection and Repair Procedure

I froze the translation-repair protocol before any repaired output existed.

The selection rule uses only model outputs. An original source translation is repaired if at least 20% of its letters fall outside the Latin range or if English function words make up less than 15% of its words. For summary translations, the function-word threshold is 12%. This selects 298 rows: 274 sources and 54 summaries, with 30 rows needing both.

Sources are split at sentence-final punctuation, including CJK and Ethiopic sentence endings, and packed greedily into chunks of at most 1,000 characters. Each chunk is translated with the unchanged original prompt and at most 1,024 new tokens. A chunk is accepted if it is non-empty, mostly in Latin script and different from its input. If it has at least 15 Latin words, it must also contain at least 10% function words. A rejected chunk is split once, at the segment boundary closest to its midpoint, and both halves are translated. Of 1,184 source chunks, 1,105 were accepted on the first pass and 79 needed the split retry. No translation reached its token limit.

F.2. Evaluation Conditions and Tests

All three conditions were computed in one run with the pinned follow-up model revision. The primary endpoint is the Spearman difference between the repaired and original conditions on all 900 rows. The matched contrast between the repaired and re-verified conditions separates the repair from changes in the runtime. Intervals use the paired language-cluster bootstrap with 2,000 replicates. Before the run, offline tests confirmed that the package reproduces the original scores and prompts exactly.

G. Additional Evaluation Protocol and Exposure Audit

In this thesis, *additional* means that the articles share no article with the two recorded development samples. It does not mean that the articles were never encountered during the project.

G.1. Sampling and Frozen Protocol

The sample was drawn from the full mFACE release (31,500 rows, 4,500 articles) after excluding the 1,461 articles of the two recorded development samples, with 20 articles per language and one summary per article. The human labels were stored separately from the model inputs. The experimental plan fixed the systems, the rule for SV-NLI's translation, the two primary endpoints and the bootstrap settings before any output existed.

All conditions of a row ran in one process, on H100 or H200 GPUs. Before the run, offline tests confirmed that the holistic prompt, the parser and the NLI aggregation reproduce the development scores exactly. A separate re-analysis of the saved records reproduces the sample selection and every reported metric and interval.

G.2. Exposure Audit

After the run, I checked my earlier working files for any of the additional articles. Thirty-one of the 900 appear in earlier selections:

- 21 in preparation sheets for manual coding that were never filled in;
- two in a list of the longest summaries;
- one in a completed 30-example English prompt comparison;
- seven in a completed 30-example source-length experiment.

I count every such article as exposed, even where a file shows only that it was selected for inspection. The evidence archive lists their row numbers in the additional sample. The sensitivity analysis repeats the comparisons on the 869 rows without these articles; it is post-hoc, changes no method, threshold or score, and the 900-row analysis remains the primary one.

The audit covers every earlier working file that I could find. It does not cover the preliminary exploration, whose rows were not recorded, translated or paraphrased duplicates, or

exposure during pretraining.

G.3. Post-hoc Controls and Within-Language Summaries

I fixed the control protocol after inspecting the original additional results but before the new fit and the control evaluation. The only fitted parameter, the calibration offset, is estimated on development data. The intervals for the unconditional mean and CalH reuse the paired language-cluster bootstrap of the main analysis. The added analysis neither changes the original systems nor replaces their primary endpoints.

Table 28 summarizes the within-language Spearman correlations. CalH keeps H's ordering in this sample, including within each language. A median of correlations is neither a pooled correlation nor a paired effect estimate. All language-level values are listed in the evidence archive.

Table 28 Descriptive within-language Spearman summaries on the additional sample. All correlations use 20 examples per language. Undefined values would be reported separately rather than set to zero. IQR is the interquartile range, from the 25th to the 75th percentile.

System	Defined	Undefined	Median	IQR	Positive
H	45	0	0.324	[0.182,0.392]	43
Mean	45	0	0.384	[0.233,0.497]	43
SV	45	0	0.352	[0.228,0.501]	44
CalH	45	0	0.324	[0.182,0.392]	43
SV-NLI	45	0	0.404	[0.285,0.527]	44
SV-NLI-base	45	0	0.325	[0.240,0.399]	43

The multiplicity sensitivity check uses the original language-cluster draws and the 1.25th and 98.75th percentiles, for the two original primary ranking contrasts only. It is a supplementary, approximate Bonferroni-style check. No adjustment is claimed for the exploratory endpoints.

The trained-verifier controls in Table 19 were defined after all earlier results had been inspected. They reuse the saved additional rows and the development scores of the bridge. They fit one additive offset per system on the 900 development examples only, clip the calibrated scores to $[0, 100]$, and use the same language-cluster bootstrap. The failure-handling check in Table 20 was added on 28 October 2026, after all other results were known. It uses the same bootstrap and seeds as the original analyses, so its first column reproduces the reported values exactly.

H. Qualitative Cases and Audit Codebook

This appendix gives the material behind the qualitative cases in Chapter 5 and the error audit in Section 4.4. Two boundaries apply to all examples below. The original-language texts and translations were not reviewed independently, so the examples show how the pipeline behaves, not how faithful the translations are. And a disagreement with a released human score does not by itself show that the human score is wrong.

H.1. Cases from Chapter 5

Section 5.5 discusses Burmese row 81, Sinhala row 614, Korean row 336 and Tigrinya row 754. The development records in the evidence archive contain their full source, summary, translated texts, extracted claims, verification responses and scores.

H.2. Audit Codebook

I wrote this codebook after the audit, based on the annotation interface and the saved assignments, so it was not fixed in advance. Section 4.4 describes how the audit was carried out.

Each case needed one primary category. A secondary category, a confidence code and a note were optional. The final file contains 50 completed assignments, 47 with confidence H and three with M, and no L code occurs. These are my own subjective codes. No rule fixed which mechanism takes precedence when several compete. Ambiguous cases involving the source or the reference stay in *other*, and records with lower confidence are included in the reported counts.

Entity/role mismatch A wrong person, entity, role, actor, attribution or location. English row 159 gets this label because the summary attributes the “greed” remark to Mark Spencer, while the source attributes it to Boris Johnson. The label concerns the speaker, not the topic around the remark.

Number/date/quantity mismatch A changed number, date, age, percentage, duration or amount. Arabic row 38 belongs here because the summary describes three police officers instead of two government officials, and it also attaches a temporal detail to the wrong attack. Several differences can occur together. The primary label does not mean that a case contains only one type of error.

Causal/relational mismatch A changed direction, cause, condition, status or policy relation. Hausa row 219 reverses Barcelona’s position. The source says that the club is not

considering re-signing Neymar, while the summary says the opposite. My note separates this reversal from a further suspected translation artefact.

Partial support Central content that the source supports, combined with an important unsupported detail. In Burmese row 95, the attack on a police outpost is supported, but the geographic details are not in the source. My note sets aside further apparent errors that I attributed to the English rendering.

Other Cases outside the four categories, and unresolved disagreements involving the source or the reference. In Arabic row 37, I found no clear inconsistency. The source appears to support the summary's statement that the Catholic Church criticized Duterte's remarks, yet the human Q2 score is zero. The case stays in *other* instead of being forced into an error category.

The exact notes, primary and secondary labels and confidence codes are in the evidence archive.

I. AI Models and Assistance

Declaration of AI use ChatGPT (OpenAI) used GPT-5.6, and Codex (OpenAI) used GPT-5.6 Sol. Claude (Anthropic) was used as Claude Sonnet in earlier phases and as Claude Opus 5.5 in the final phase in October 2026.

Language support Claude and Codex were used for grammar correction, spell checking and sentence-flow editing of the text I had written. In the final phase, I also used Claude Opus 5.5 to redraft passages throughout the thesis, including the abstract, for clarity and concision. I edited all redrafted text manually, checked every statement and number against my results, and made the final wording decisions. I did not adopt any AI text without manual revision.

Programming assistance I specified the analyses, including the metrics, statistical tests and comparisons to be performed. Following these specifications, Claude and Codex wrote and debugged the Python code, and ChatGPT provided programming assistance for the KLUE transfer test. This assistance covered the training and inference scripts, the numerical calculations, the reproduction scripts and the packaging of the evidence archive, as well as the run protocols and the instructions for running jobs on the computing cluster. For the KLUE transfer test, ChatGPT also suggested prompts and design choices, which I reviewed before deciding on them. I reviewed the code and checked its outputs, and I interpreted the results and decided on the conclusions. Apart from these KLUE suggestions and the methodological assistance listed below, AI assistance with programming was limited to implementing the specified procedures.

Methodological assistance ChatGPT was the assistant in the audit and preference-pair steps of Section 3.1: it proposed refinements to the candidate error categories, gave a second reading of audit cases, and helped construct and quality-check the Korean preference pairs. Claude assisted with the audit of the stored translation outputs, the English-likeness heuristic and the translation-repair procedure. Section 3.1 describes how the final labels, pairs and procedures were decided.

Author review and responsibility I directed the research and made the final methodological and experimental decisions. I reviewed and revised all adopted AI-assisted text, code and documentation, and I am responsible for their final selection and use.

J. Evidence Archive and Reproducibility Scope

This appendix describes the submitted evidence archive and sets out what it can and cannot reproduce.

J.1. Evidence Archive

The accompanying archive `Thesis_Evidence_2026-10-24_v1.zip` (identifier TE-20261024-v1) is submitted together with the thesis and is not a public repository. Its `EVIDENCE_MANIFEST.json` lists SHA-256 hashes and paths, and `README.md` gives the reproduction commands. The reproducibility index in the manifest maps the names used in this thesis to the exact files and lists the seeds, model revisions and software environments recorded for each run. Table 29 summarizes the role of the main records, and Table 30 summarizes what the archive contains and where its limits lie.

The separate code package `Thesis_Code_2026-10-25.zip` contains the scripts, analysis entry points and documentation, prepared for publication in a repository. Its code map distinguishes the saved-score analysis, which can be run directly, from the original runners, which need external inputs and model weights.

The original mFACE publication and release keep their attribution and licence. Reconstructing the samples requires the full release, but the selected inputs are included for direct score analysis. Where archived files use the term “pre-registered”, it refers to internal protocol freezing, not to external registration.

Table 29 Guide to the principal reproducibility records. Exact identifiers are kept in the evidence index.

Record	Role and scope
Frozen development input	Defines the main 900-example sample; distinct from the earlier atomic sample and the additional article sample.
Development verification output	Preserves translations, extracted claims and verification outputs for the development sample.
Language-resource mapping	Records the XL-Sum training counts used for resource groups.
Korean DPO records	Identify the preference adapter, evaluation protocol and original/reversed candidate orders.
Supervised verifier and bridge records	Identify the separately trained verifier and its use in the selective pipeline.
Translation-repair and additional-evaluation records	Identify the frozen follow-up protocols and their inputs.

Table 30 Contents and limits of evidence archive TE-20261024-v1.

Item	Identifiable records
Evaluation data	Released mFACE identity and checksum; selected row and article IDs; development and additional inputs, references and saved outputs.
Models	Pinned follow-up base revision and adapter checksums; early-run revision not recorded; weights not included.
Execution	Scripts, plans, seeds, frozen protocols and recorded environments; CPU analysis environment recorded separately.
Analysis	Commands and saved-score code for the principal pipeline comparisons, post-hoc controls including the trained-verifier controls, within-language summaries, multiplicity sensitivity, the exploratory checks of Chapter 8, and the failure-handling and scale checks of Chapters 4 and 7.
Audit	Annotation interface, assignments, notes and confidence codes of the 50-case audit.

J.2. Remaining Limits of Reproduction

The computational checks recompute the reported results and sample joins from the saved outputs. They do not regenerate the original model outputs, and they do not validate the translations or the human references. Within this scope, the following records are incomplete:

- The early main runs did not record the exact model revision and software environment; the follow-up runs record both.
- Without the generated token IDs and stopping metadata, the stopping reasons of the early generations, including the seven outputs near the limit (Appendix B.6), cannot be reconstructed.
- The procedure that produced the tokenizer-fragmentation counts, that is, the script, the tokenizer and its revision and the exact text selection, was not recovered, so the counts cannot be regenerated.
- The XL-Sum dataset revision behind the training counts was not recorded.
- The saved records give no complete chronology of the exploratory decisions, including the timing of the manual audit and of the Korean preference design. The rows and model of the preliminary exploration that motivated the focus on Q2 were not recorded.